The CRTC6845, the Gate Array and how they’re affecting the monitor and the interruptions of the Amstrad CPC

Abstract :

This document describes how everything works, and also a way to exploit the Reg3/CTM644 features on two not-known ways.

Acknowledgements :

This document has been written by the Cheshirecat, cheshirecat@dup.as, with the great help of all the members of the Impact discord (GurneyH, Kris, Sid...). Many thanks to all of them !

Table of contents

Higher nibble & VBL....3
Lower nibble : HBL, screen shifting, video mode change & interruptions....4
- Further Hsync considerations....5
  - On the MC6845P (CRTC2) :....5
  - On the UM6845R (CRTC1) :....5
  - On the (CRTC3) :....6
- Videomode switching....7
- Interruptions....7
- Horizontal scrolling using the HBL....8
  - The test program....8
  - Standard switching between values 4, 5 & 6....10
    - 4→5....10
    - 4→6....10
    - 5→4....11
    - 5→6....11
    - 6→4....12
    - 6→5....12
  - Programming tricks....13
    - Fast transition from 6 to 5....13
    - Smaller shifting : Reg3 set to 5,5....14

Questions which still need an answer....15
- Strange things happen during the VBL....15
- Loss of synchronization....15

Test programs....16

Higher nibble & VBL

The 4 higher bits of the Reg3 might be used, depending on the version of the CRTC6845. The Amstrad CPC 6128 firmware sets this nibble to 8.

- On the CRTC 0 (HD6845 from Hitachi) : The 4 higher bits are used to set the height of the VBL, expressed in scanlines. Setting this nibble to 0 will produce a 16 scanlines Vsync.
- On the CRTC 1 (UM6845R from UMC) : The 4 higher bits are not used.
- On the CRTC 2 (MC6845P from Motorola) : The higher bits are not used, as noted in the documentation of the MC6845.
- On the CRTC 3 (Amstrad 40489 : The 4 higher bits are used to set the height of the Vsync, expressed in scanlines. Setting this nibble to 0 will produce a 16 scanlines Vsync.
- On the CRTC 4 (Amstrad 40226 & 40235) : The 4 higher bits are used to set the height of the VBL, expressed in scanlines. Setting this nibble to 0 will produce a 16 scanlines Vsync.

Note : Some documentations you could find on the Internet are talking about doubling the Vsync if the bit 7 of the Reg3 is set to 0, and « normal » length of VBL if set to 1 on the CRTC 0, 3 and 4. This is true but incomplete. True because by setting the higher nibble of the Reg3 to 8 (i.e bit 7 to 1) will lead the CRTC to generate a 8 scanlines VBL, while setting it to 0 (bit 7 to 0) will generate a 16 scanlines Vsync. It’s an incomplete information because the bits 6, 5 and 4 could allow a more precise control of the length of the Vsync.

The test program test_reg3_Vsync_size allows us to measure the length of the Vsync signal (In scanlines). It displays two column of hexadecimal numbers : The first number represents the number of NOPs it has counted during the Vsync (15-24 NOPs of margin of error), the second one is the number of scanlines it represents. The first line is for higher nibble=0, the second, third, fourth and fifth are not tested because it might desynchronize the monitor, sixth line is for higher nibble=5, seventh for 6, and so on up to 15. The resulting screen looks like that :



On the CRTC Type 0 (Unfortunately we weren't able to know its manufacturer reference), we get the following values :

```
0404 10

0148 05
0184 06
01C0 07
0206 08
0242 09
0288 0A
02C4 0B
0300 0C
0346 0D
0382 0E
03C8 0F
```

As you can see, each value of the higher nibble changes the height of the Vsync signal : 0 generates a 16 scanlines Vsync, 5 generates a 5 scanlines Vsync, and so on.

On the MC6845P we see only lines of « 0404 10 ». This is the proof that the higher nibble is not used on the CRTC and the Vsync signal always lasts for 16 scanlines.

Lower nibble : HBL, screen shifting, video mode change & interruptions

The HSYNC/VSYNC signals issued by the CRTC are received by the Gate Array which mixes them in order to generate the Csync (composite sync) signal which will be sent to the CTM644.

The Gate Array waits for 2µs (i.e 2 NOPs or 2 chars) before sending the Hsync (mixed with the Vsync signal, into he Csync) to the CTM. This means values lower or equal to 2 won't generate any Hsync signal to the monitor. But the G.A takes this signal in account to do some processings, including the videomode changings. This implies that multimode on a scanline is not « dirty » on the point of view of the CTM since it won't receive any Hsync signal when Reg3=2.

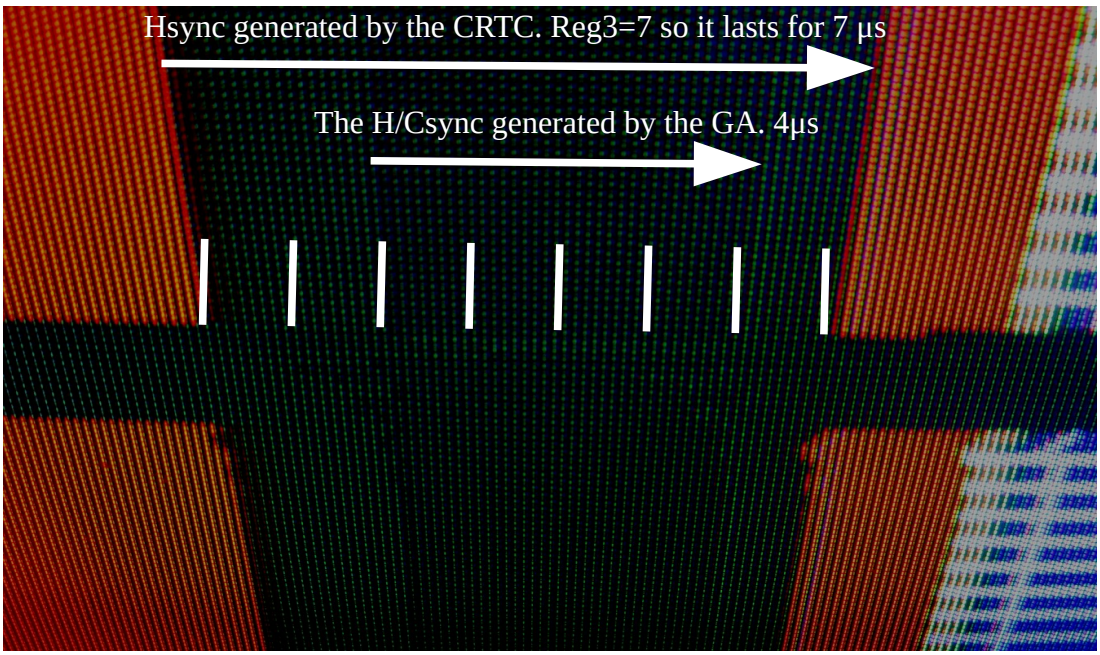
The Gate Array will also limit the duration of the horizontal synchronization in the Csync signal to 4µs. This means there is no interest into setting the Reg3 to a value greater than 6 (2µs of « latency » plus 4µs of H/Csync). If you set the Reg3 to 3 the H/Csync signal will last for only 1µs, which is not sufficient to get a horizontal synchronization on most of the monitors.

The H/Csync could be seen on a CTM644 (You have to push the luminosity to its maximum), thanks to the following piece of code (test_reg3_HCsync). One could also see the waggling which starts a few lines after the start of the VBL and lasts for around 12 scanlines after it.

```
    outcrtc 6,40
    outcrtc 1,16

    outcrtc 7,20
    outcrtc 2,24
    outcrtc 0,31
    outcrtc 3,7

loop:
    jr loop
```



As you could see, the values of interest for the lower nibble of the Reg3 are only 2 (videomode switching without generating any H/Csync signal), 4, 5 and 6 (for scanlines shifting) when you want to deal with the CTM. Higher values are nevertheless interesting for delaying the interruptions (The interruptions are ticked by the Gate Array at the end of the Hsync signal).

Further Hsync considerations

We have discovered the Hsync signal is not fired at the start of a char, thanks to the test_reg1_reg3_Hsync program.

```
    outcrtc 1,31
    outcrtc 6,40

    outcrtc 7,20
    outcrtc 2,16
    outcrtc 0,31
    outcrtc 3,7
```

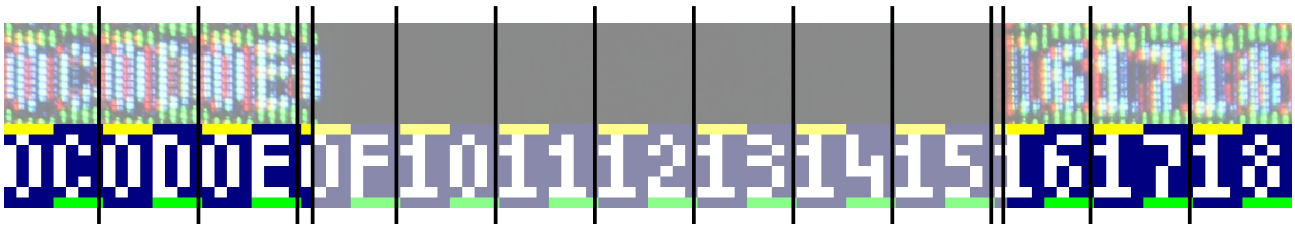
We also discovered the CPC+ Monitor can’t manage several HBLs on the same scanline, so we had to change our test program : It’s now test_reg1_reg3_HSync2

```
    outcrtc 1,31
    outcrtc 6,40

    outcrtc 7,20
    outcrtc 2,16
    outcrtc 0,31

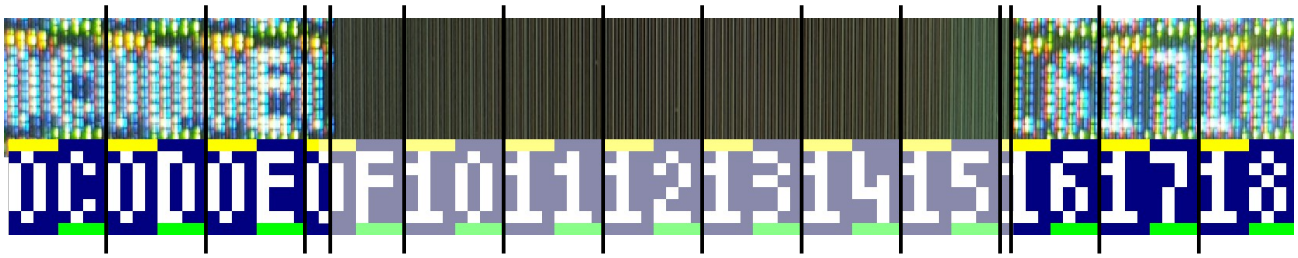
loop:
    outcrtc 3,7
    delay 32-14
    outcrtc 3,2
    delay 32-14-3
    jr loop
```

On the MC6845P (CRTC2) :



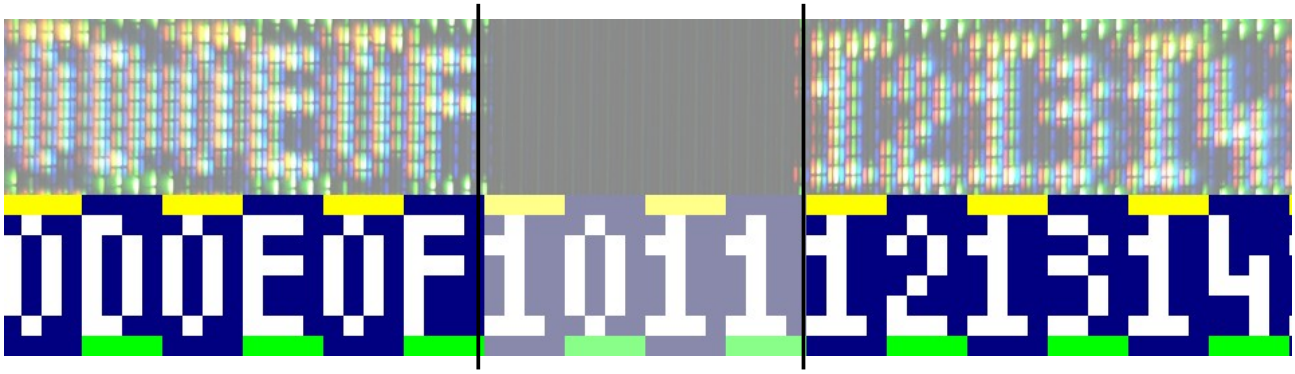
test_reg1_reg3_Hsync : The Hsync signal starts 1 MODE 1 pixel (1/8 of a microsecond) after the start of the 16th char (value given by the Reg2), and ends at the (Reg2+Reg3)th + 1/8 of a microsecond.

On the UM6845R (CRTC1) :



test_reg1_reg3_Hsync : The Hsync signal starts 2 MODE 1 pixels after the start of the 16th char and ends at the (Reg2+Reg3) + 1/8 of a microsecond

On the (CRTC3) :



test_reg1_reg3_Hsync2 : The Hsync signal starts when the (Reg2)th char is over and ends when the (Reg2+Reg3)th char is over

Videomode switching

Interruptions

Horizontal scrolling using the HBL

This feature is not actually offered by CRTC6845 nor the Amstrad CPC itself, it’s a feature of it’s monitor, the CTM644. These scanlines shiftings are due to the tolerance values of the monitor. Therefore, if you use a different monitor than the one bundled with the CPC, it may not work (and some tests have showed it’s not). Moreover, we have expressed the right/left shiftings in pixels, but the CTM644 doesn’t manage pixels in themselves. It only processes a video signal.

The test program

We wrote a small program (test_reg3_scanlines_shifting) to explore the feature. This code extensively uses macros :

OUTCRTC puts a value into a CRTC register (lasts for 14 NOPs), SETBORDER sets the border colour, SETINK sets a colour to an ink number, DELAY waits for a given number of NOPs

```
Frame:
    ei
    ld b,#f5
.loop:
    halt
    in a,(c)
    rra
    jr nc,.loop

    outcrtc 3,6

    ld bc,#bd04
    halt ; 1
    out (c),c : setink 0,12 : setink 0,20

    di
    delay 16*64-14*3

    outcrtc 3,6 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,4 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,5 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,4 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,5 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,6 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,5 : setink 0,12 : setink 0,20 : delay 16*64-14*3
    outcrtc 3,6 : setink 0,12 : setink 0,20 : delay 16*64-14*3

    outcrtc 3,4 : setink 0,12 : setink 0,20 : delay 64+64-14*3
    outcrtc 3,5 : setink 1,12 : setink 1,11 : delay 14*64-14*3

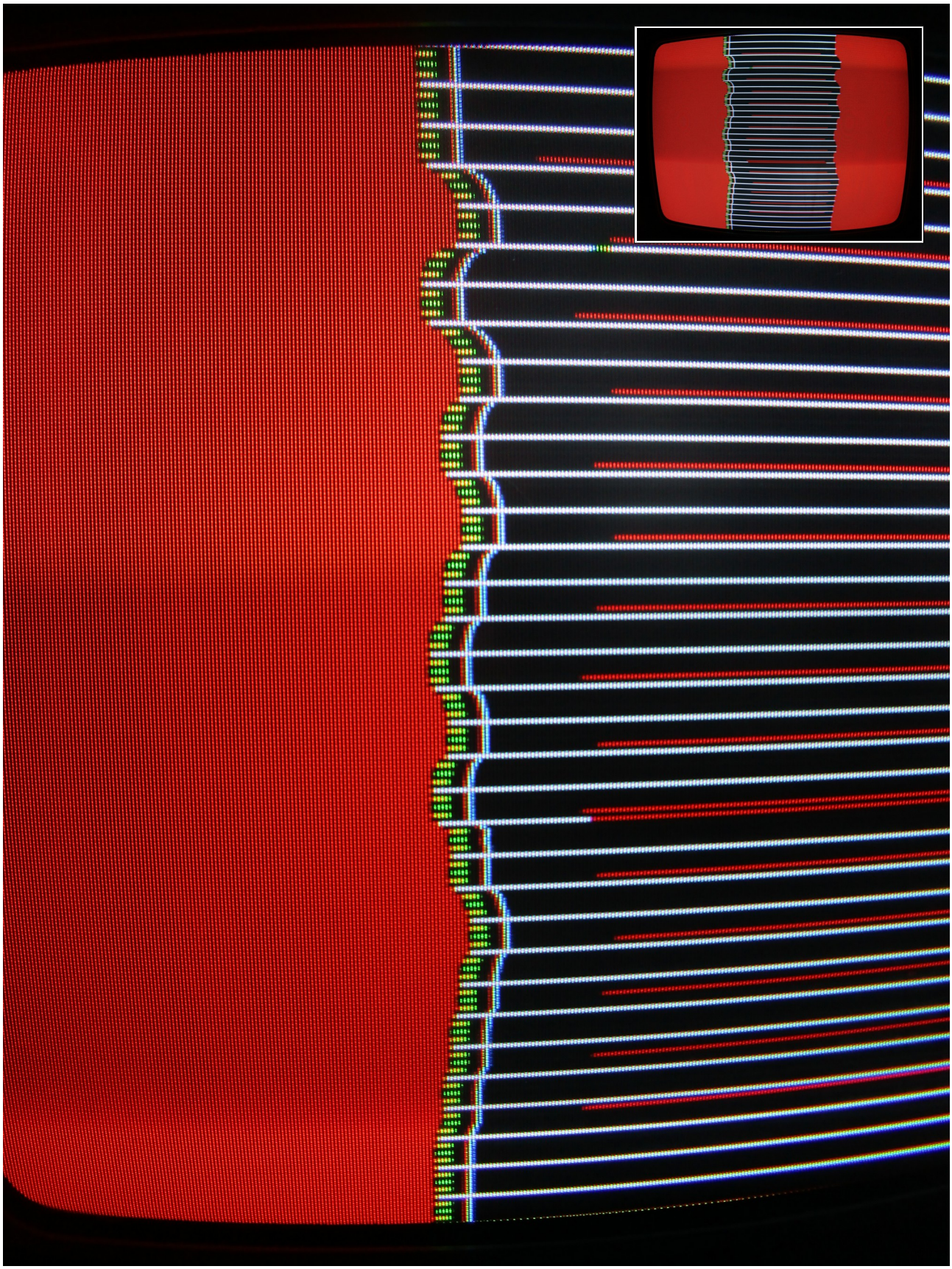
    outcrtc 3,4 : setink 0,12 : setink 0,20 : delay 16*64-14*3

    outcrtc 3,5 : setink 0,12 : setink 0,20 : delay 64-14*3
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,4 : delay 64-14
    outcrtc 3,5 : setink 0,12 : setink 0,20 : delay 16*64-14*3

    outcrtc 3,6 : setink 0,12 : setink 0,20 : delay 64-14*3
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : delay 64-14
    outcrtc 3,5 : delay 64-14
    outcrtc 3,6 : setink 0,12 : setink 0,20

    jp Frame
```

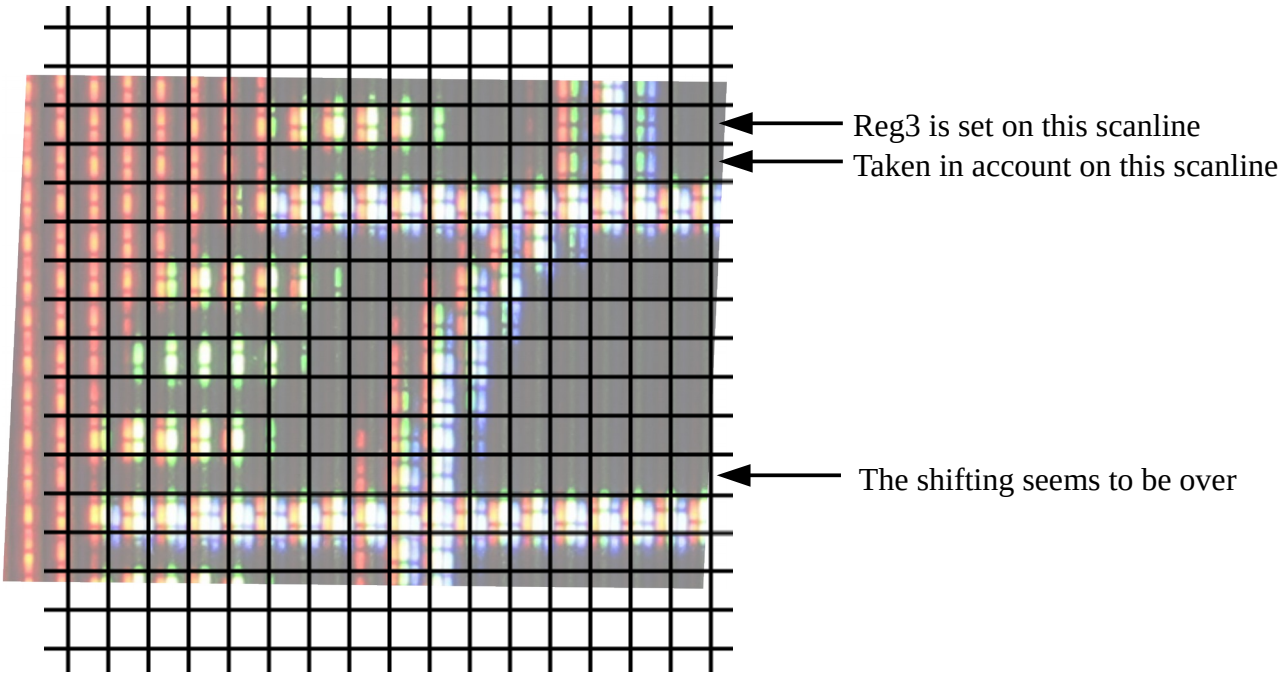
The code produces this display on the CTM644 :



Standard switching between values 4, 5 & 6

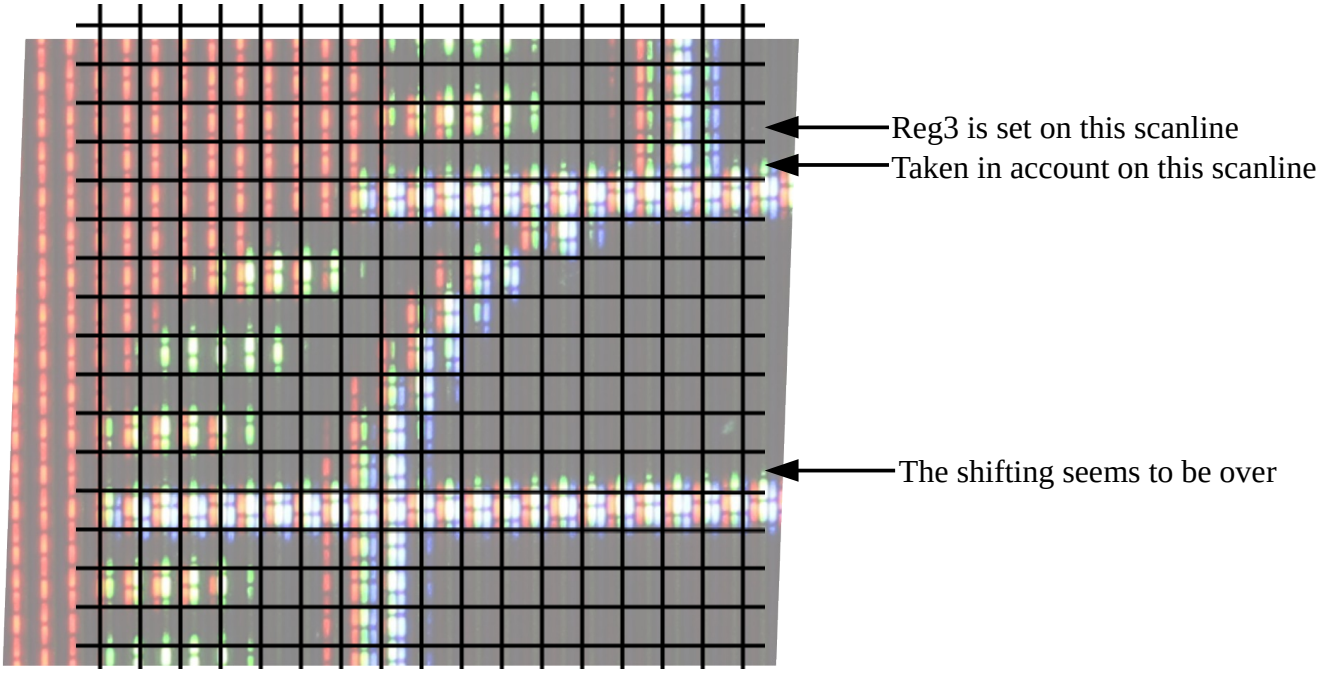
It seems that the shape of the curve also depends on the direction (to the left or to the right) of the shift.

4→5



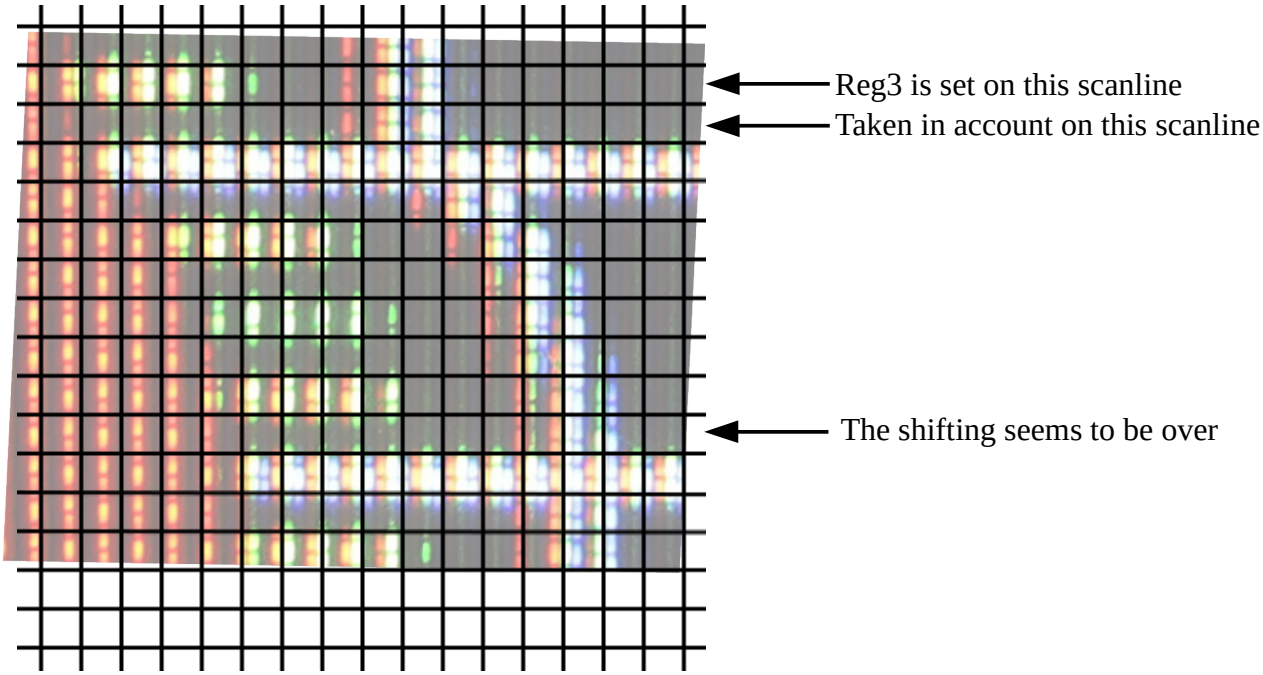
Shifting of 4,5 pixels to the left. 9 scanlines needed.

4→6



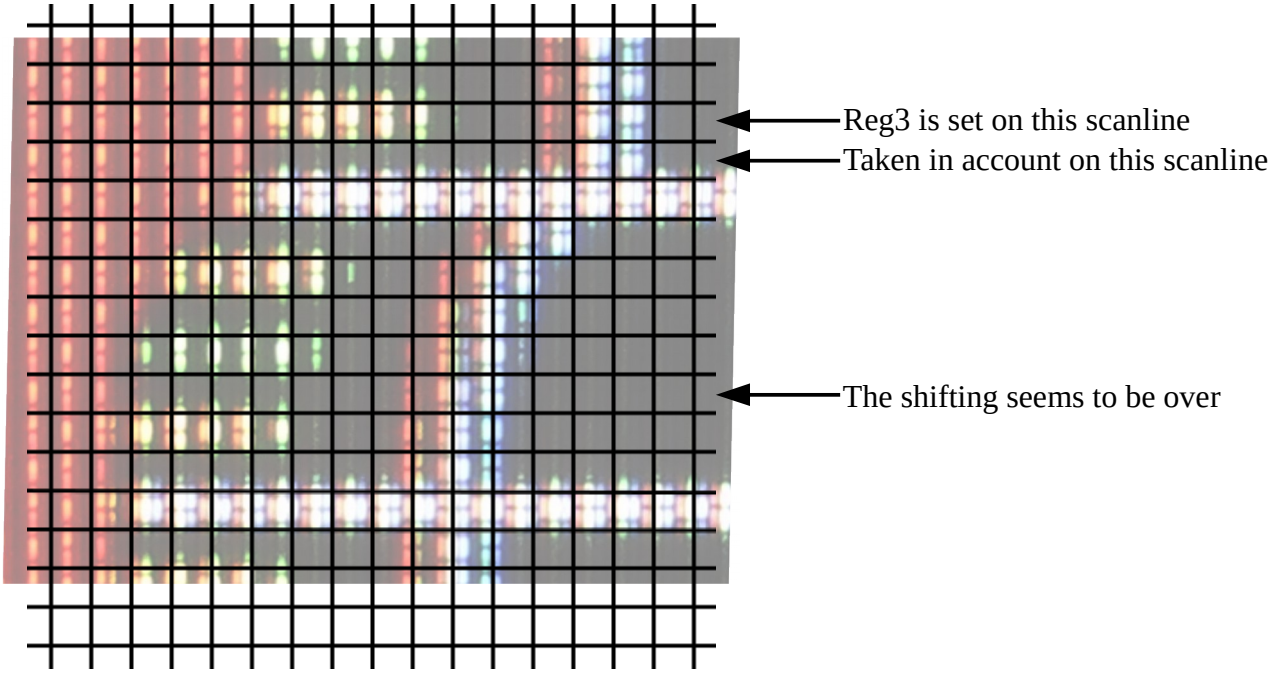
Shifting of 8 pixels to the left. 9 scanlines needed.

5→4



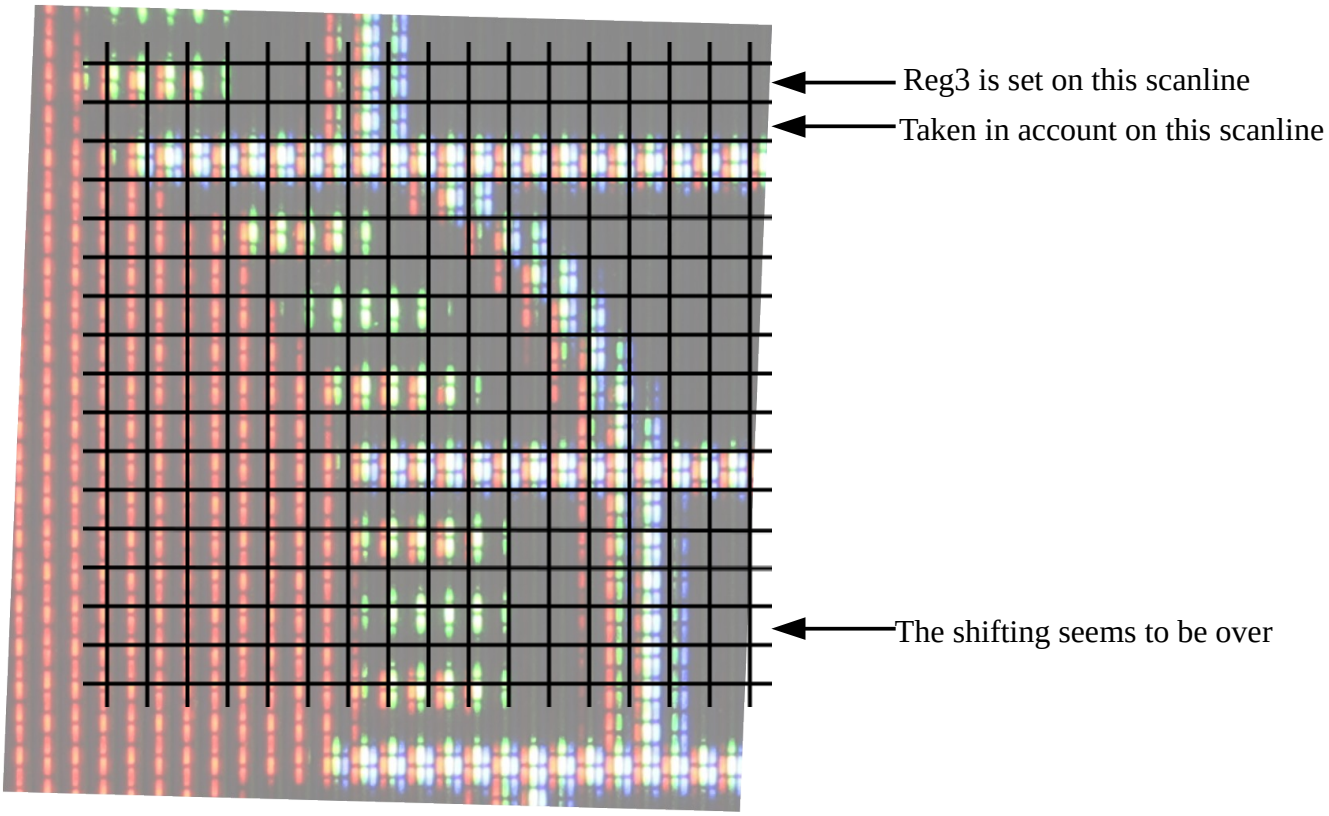
Shifting of 4,5 pixels to the right. 9 scanlines needed.

5→6



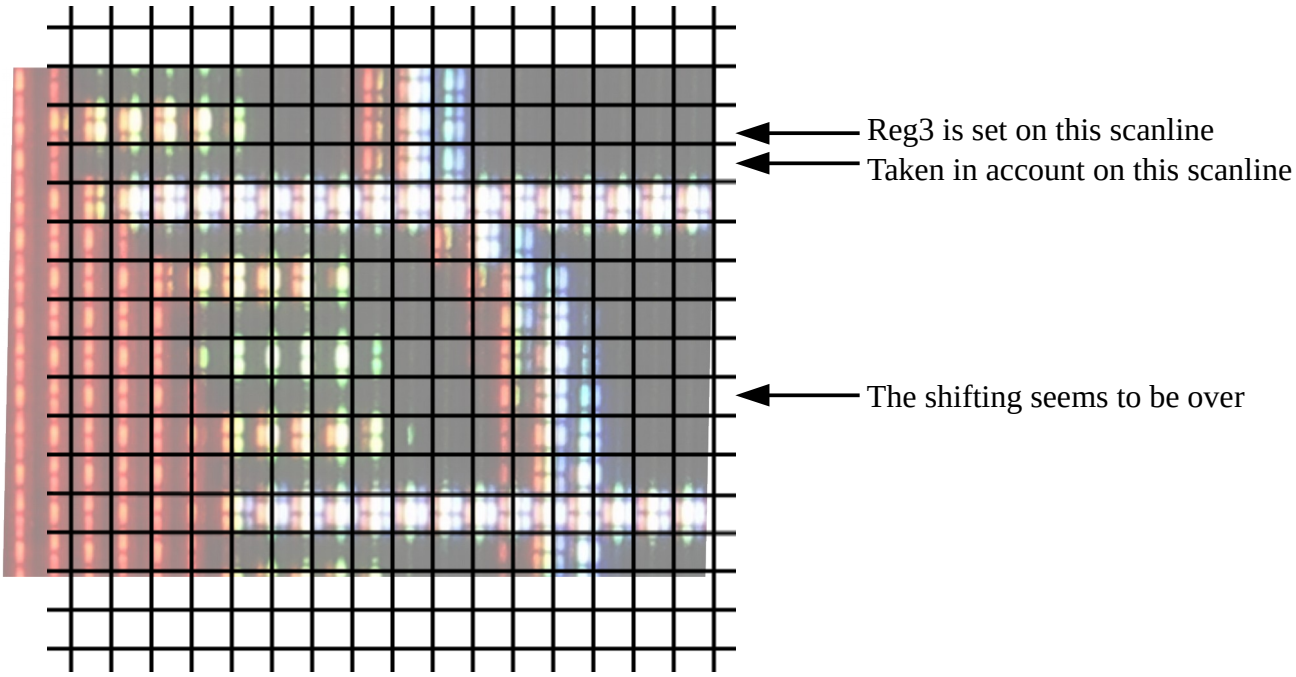
Shifting of 3,5 pixels to the left. 7 scanlines needed.

6→4



Shifting of 8 pixels to the right. 14 scanlines needed.

6→5



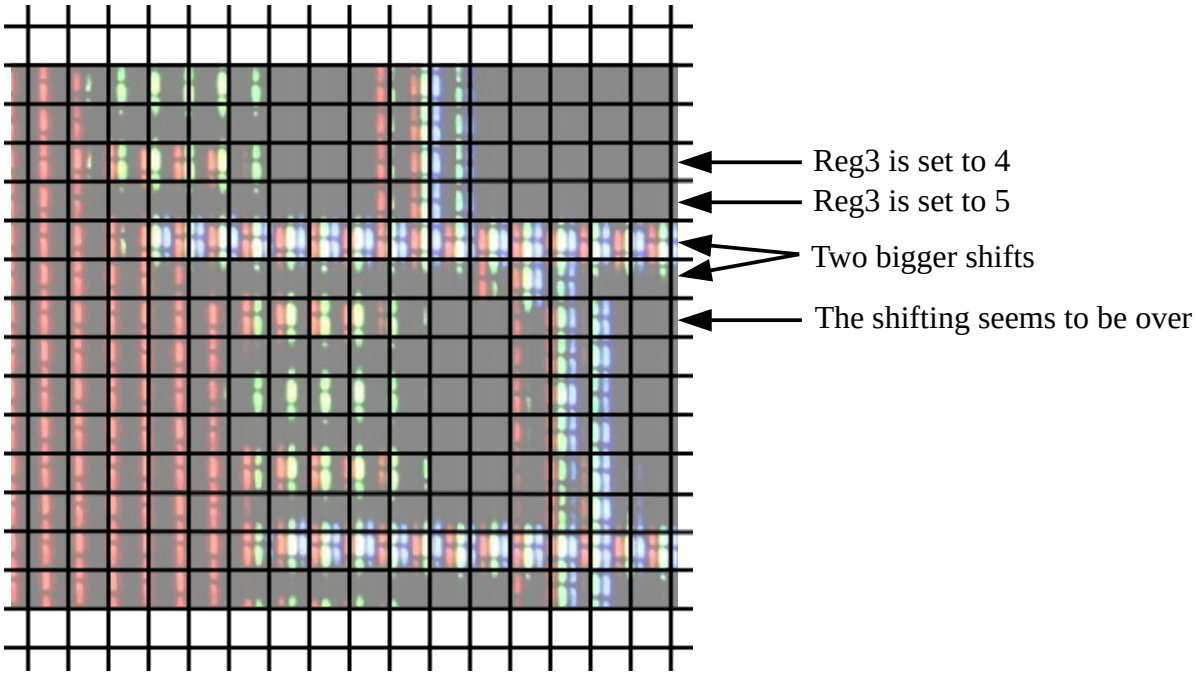
Shifting of 3,5 pixels to the right. 7 scanlines needed

Programming tricks

Fast transition from 6 to 5

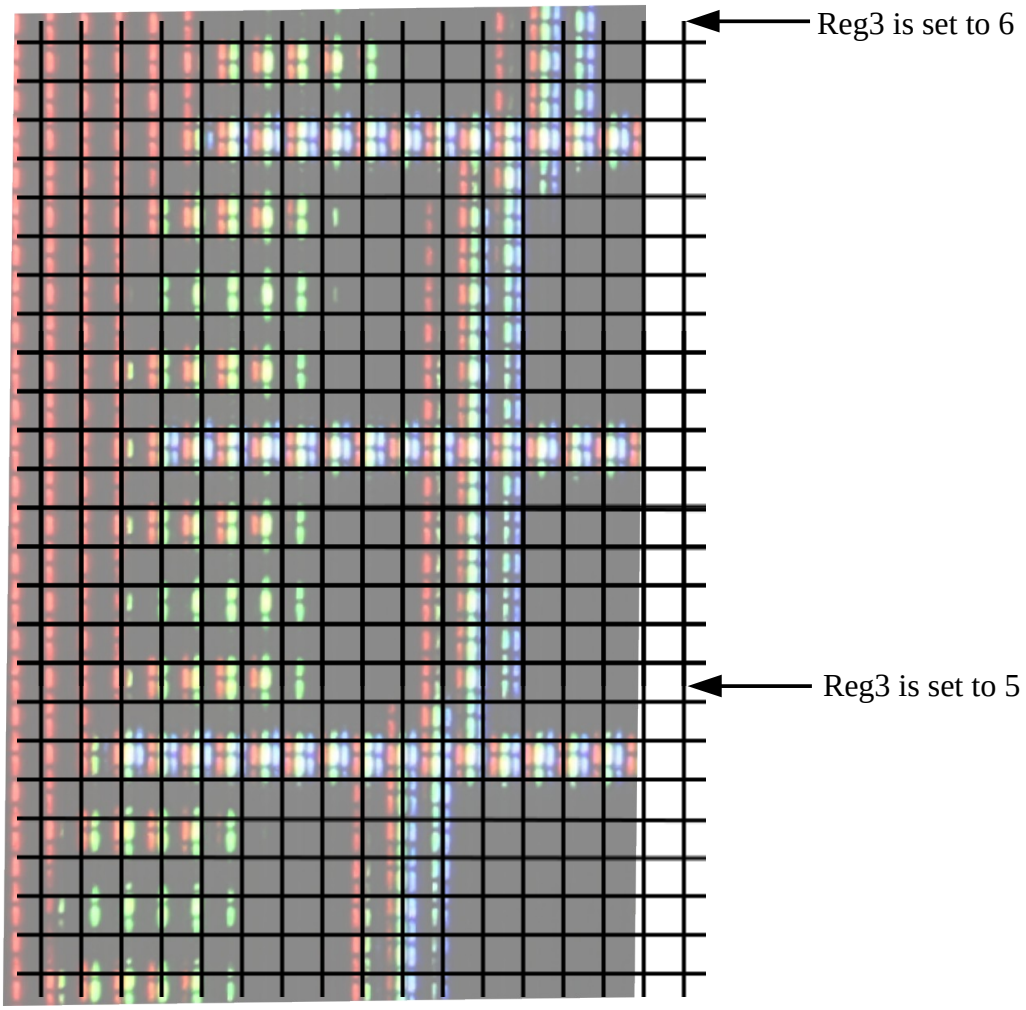
This programming trick exploits the fact that the curve is different between each value of Reg3. First we set the Reg3 to 4 for one scanline in order to force a faster shift, then set it to 5. This way the shifting lasts for only 4 scanlines instead of 7.

One could see a very small « bouncing back » 2 or 3 scanlines after.



Smaller shifting : Reg3 set to 5,5

Alternate value of the Reg3 between 6 and 5 on each scanline produces a surprinsingly very stable shift, which lets you get an intermediary switching. On the point of view of the user, we have a switching of half a byte. The downside is it’s really time-consuming for the CPC.



The same trick could be done in order to obtain a kind of « 4,5 » value, by setting alternatively the Reg3 to 4 & 5.

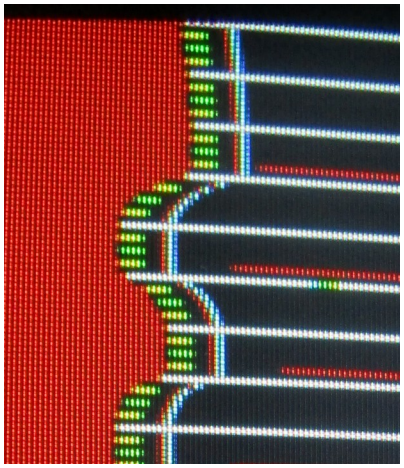
Questions which still need an answer

Strange things happen during the VBL

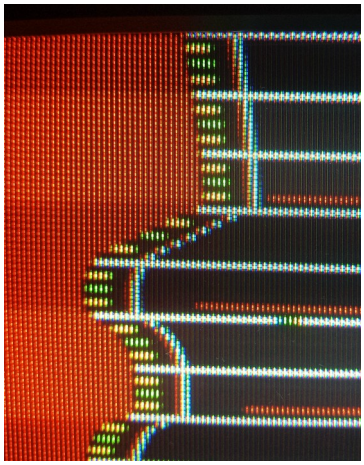
When I tested the program, I dicovered that if I change the reg3 from 6 to 4 during the VBL, the scanlines are moved approximately one char to the right from where I expected. Changing the reg3 to 6 afterwards puts the scanlines at the ‘good’ place. Setting Reg3 to 4 then puts the scanlines at the place we expected.

You’ll also notice the scanlines « derivate » from left to right when the Reg3 is set to 4 too early into the frame.

When the Reg3 is set to 4 after the VBL



When the Reg3 is set to 4 during the VBL



Loss of synchronization

When one puts the Reg3 to 3 or 2, the scanlines are shifted to the right, the synchronization is lost. You could get the horizontal synchronization back by setting a good value the Reg3 during the frame. It could be interesting to study this lose/recovery of synchronization.

Test programs

Summary of all the test programs, and on which CRTC they have been tested. **If your CTRC has not been tested**, please run the corresponding test and take a picture of the screen it generates, then send it to cheshirecat@dup.as.

	CRTC 0			CRTC 1	CRTC 2			CRTC 3	CRTC 4
Program	**HD6845S**	**HD6845SP**	**UM6845**	**UM6845R**	**MC6845**	**MC6845P**	**UM6845S**	**Amstrad 40489**	**Amstrad 40226**
test_reg1_overflow_border									
test_reg3_HCsync						Claire			
test_reg1_reg3_HSync				Kris		Claire		Kris/Sid	
test_reg1_reg3_HSync2								Kris	
test_reg3_scanlines_shifting						Claire			
test_reg3_multimode									
test_reg3_VSync_size	Gurneyh ?	Gurneyh ?	Gurneyh ?			Claire			